

psac: portfolio risk on self-adjusting computation

A walkthrough of the portfolio demo — live market data, incremental risk measures, policy-as-SAC access control, cost attribution, provenance-explained reports, and what-if scenarios, with every core mechanism backed by a Lean 4 theorem.

Repository: github.com/egao1980/psac · demo entry point: (psac:run-portfolio-demo) in src/portfolio.lisp

1. The idea in one paragraph

psac is a self-adjusting computation (SAC) runtime: computations declare which *modifiabiles* (“mods”) they read, and when an input changes, change propagation re-runs exactly the affected sub-computations — in dependency order, with an equality cutoff. Because the dependency trace is a first-class data structure, three things fall out almost for free: **cost attribution** (each re-run is billed to the principals whose writes caused it), **provenance** (the trace *is* the derivation DAG, so “why is this number what it is?” is a graph walk), and **access control as SAC** (policy facts are mods too, so revocation is just change propagation). The portfolio demo exercises all of it on one realistic workload.

2. Cast and book

Three principals:

Principal	Role	Access
feed	market-data source	writes prices; blamed for tick-driven recomputation
alice	risk manager, group :risk	everything: raw prices, per-position detail, firm aggregates
bob	desk B trader, group :desk-b	his desk’s aggregates only

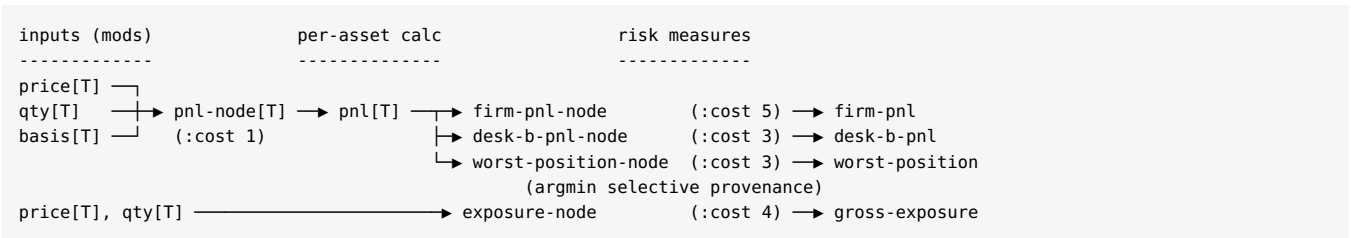
The book (prices in integer cents; source cost = predefined per-update cost of that feed):

Ticker	Price	Qty	Basis	Source cost
AAPL	19000	100	18000	2
MSFT	41000	50	40000	2
GOOG	17500	−30	18000	2
BTC	6500000	2	6000000	5
EURUSD	10850	1000	10800	1

Desk B holds AAPL and MSFT.

3. Architecture

The adaptive dependency graph built by make-universe (T ranges over tickers):



Policy lives in the same graph, one stratum earlier (stratum 0 quiesces before any data-stratum work, so access decisions are always consistent with the latest policy):

```

stratum 0 (policy)                                stratum 1 (data)
-----
member?(bob,desk-b) ─┐
grants(desk-b,desk-b) ┤─▶ allowed-node(bob,desk-b) ─▶ allowed?(bob,desk-b) ─▶ guards views

```

Underneath, the same runtime provides level-synchronous parallel propagation (propagate-parallel!) and fork-join inside computations (par, par-map) — see section 11, proof-backed per section 12.

4. Building the book (code)

Per-asset P&L is one adaptive node over three mods:

```

(register-read (list price-mod qty-mod basis-mod)
  (lambda (p q b) (write! pnl-mod (* q (- p b)))))
:name (format nil "pnl-node[~a]" ticker)
:cost 1)

```

Risk measures carry predefined calculation costs; the worst position declares *selective provenance* — only the argmin input explains its value:

```

(register-read pnl-mods
  (lambda (&rest vals)
    (let ((m (reduce #'min vals)))
      (write! worst m)
      m))
  :name "worst-position-node" :cost 3
  :provenance (lambda (result vals mods-read)
    (list (nth (position result vals) mods-read))))

```

Access policy is data, not code — group membership and grants are stratum-0 mods:

```

(admit! "alice" :risk)
(admit! "bob" :desk-b)
(dolist (class '(:marketdata :firm :desk-b))
  (grant-class! :risk class))
(grant-class! :desk-b :desk-b)

```

5. Update-driven billing: the market ticks

Market events are ordinary writes blamed on their author:

```

(tick! u "AAPL" 19500) ; with-principal "feed" → write! → propagate!
(book-trade! u "GOOG" -50 17800 :principal "alice")

```

Demo output:

```

--- market opens: feed ticks ---
propagation bill (blamed on writers): (("feed" . 17))
--- alice books a trade ---
propagation bill: (("alice" . 5))

```

Where the 17 comes from — the AAPL and BTC ticks dirty two P&L nodes, and everything downstream re-runs once at its predefined cost:

Re-run node	Cost
pnl-node[AAPL]	1
pnl-node[BTC]	1

Re-run node	Cost
firm-pnl-node	5
desk-b-pnl-node	3
exposure-node	4
worst-position-node	3
total, billed to feed	17

Alice's trade shows the **equality cutoff**: her GOOG amendment (qty $-30 \rightarrow -50$, basis $18000 \rightarrow 17800$) happens to leave GOOG P&L at exactly 15000, so `pnl[GOOG]` doesn't change value and the aggregates never re-run — only `pnl-node[GOOG]` (1) and `exposure-node` (4), hence the bill of 5. Costs are split among co-blamed principals by integer division with the remainder to the lowest id — exactly conserved, and remainder-to-lowest is itself proved (`charge_conserve / bill_conserve / blame_head_lowest` in Lean).

6. Access control: what Bob can and cannot see

Every view is catalogued with a resource class; request consults the self-adjusting `allowed?` mod:

```
alice requests :FIRM-PNL -> 1065000 (charged 23)
alice requests :WORST-POSITION -> 15000 (charged 21)
bob requests :DESK-B-PNL -> 200000 (charged 10)
bob requests :FIRM-PNL -> :DENIED (charged 1)
bob requests (:PRICE . "AAPL") -> :DENIED (charged 1)
bob requests (:PNL . "MSFT") -> :DENIED (charged 1)
```

Bob gets his desk aggregate (AAPL + MSFT P&L = 200000) but neither raw prices, per-position detail, nor firm-wide numbers. Revocation is eager and exact: (`revoke! "bob" :desk-b`) flips one `stratum-0` mod, propagation re-runs precisely the computations whose authority depended on it, and Bob's next request answers `:denied`.

7. Request-driven billing: pay for your slice

A request charges a flat API fee plus the **provenance slice** of the requested view — the predefined source-data costs of every input it depends on, plus the `calc` costs of every node in its derivation:

```
(defun slice-cost (universe mod)
  (multiple-value-bind (inputs nodes) (derivation-slice mod)
    (+ (loop for m in inputs sum (gethash m (universe-data-costs universe) 0))
       (loop for n in nodes sum (rnode-cost n)))))
```

Worked charges from the run above (fee = 1):

Request	Sources	Calc nodes	Charge
alice :firm-pnl	$2+2+2+5+1 = 12$	$5 \times 1 + 5 = 10$	23
alice :worst-position	12	$5 \times 1 + 3 = 8$	21
bob :desk-b-pnl	$2+2 = 4$	$2 \times 1 + 3 = 5$	10
bob (anything denied)	—	—	1

Bob's slice is smaller *by construction* — he pays less because his view provably depends on less. Ledger after the session: `((("alice" . 44) ("bob" . 13))`.

8. Alice's report: every number explains itself

`(risk-report u :shock-ticker "AAPL" :shock-bps -1000)` produces:

```
=== Portfolio risk report ===
firm P&L: 1065000 gross exposure: 28525000 worst position P&L: 15000 desk-B P&L: 200000
```

```
-- P&L attribution --
AAPL: qty=100 basis=18000 price=19500 -> P&L 150000
MSFT: qty=50 basis=40000 price=41000 -> P&L 50000
GOOG: qty=-50 basis=17800 price=17500 -> P&L 15000
BTC: qty=2 basis=6000000 price=6400000 -> P&L 800000
EURUSD: qty=1000 basis=10800 price=10850 -> P&L 50000

-- worst position, explained (selective provenance) --
determined by: basis[GOOG], qty[GOOG], price[GOOG]

-- firm P&L support (every input it depends on) --
basis[AAPL], basis[BTC], basis[EURUSD], basis[GOOG], basis[MSFT], price[AAPL],
price[BTC], price[EURUSD], price[GOOG], price[MSFT], qty[AAPL], qty[BTC],
qty[EURUSD], qty[GOOG], qty[MSFT]

-- last update: causal chain --
pnl-node[GOOG] re-ran (blame: alice) cost 1 -> wrote pnl[GOOG]
exposure-node re-ran (blame: alice) cost 4 -> wrote gross-exposure

-- counterfactual --
if AAPL moved -1000bps (19500 -> 17550): firm P&L would be 870000 (now 1065000)
```

Reading it section by section:

- **Worst position, explained** — the selective-provenance declaration slices the explanation to the argmin positions (all of them, under ties): GOOG's three inputs, not all fifteen. A selective slice *explains the current value*; it is not an influence bound — any position dropping below the current worst would change the number. The Lean-proved guarantee (support_sound) applies to the full, non-selective support, as printed for firm P&L below: inputs outside *that* set cannot change the value.
- **Causal chain** — the last propagation verbatim: what re-ran, on whose blame, at what cost. Note it records the cutoff story from section 5 (only two nodes re-ran).
- **Counterfactual** — probe runs one incremental propagation forward and one back; the world, bills, and logs are untouched afterwards.

9. What-if scenarios: tagged, private, as-if

The multi-write generalization: a **scenario** is a named batch of hypothetical writes, applied, propagated, observed, and rolled back — with its own bill charged to its owner:

```
(what-if (list (cons aapl-price 18000)
               (cons msft-price 39000))
         (list (universe-firm-pnl u))
         :tag "crash-test" :owner "alice")
```

On a three-asset book (AAPL/MSFT/GOOG as in section 2, GOOG ticked to 17600, base firm P&L 162000) this answers **-38000**, bills 17 cost units to alice *on the scenario record* ((scenario-bill-alist "crash-test")), and afterwards the base universe, **last-bill**, and the update log are byte-identical to before. Scenarios nest LIFO and roll back on non-local exit too.

10. Dynamic membership: a growing book

make-universe fixes every read set at build time — psac's **static-topology assumption**: node read sets never change, heights are computed once, and a topology change is either *additive* (new nodes over live mods) or done by building a fresh universe (universes coexist in one graph). The dynamic book (make-dynamic-book / add-asset! in src/portfolio.lisp) shows the additive pattern: firm P&L lives in an adaptive-forest — a binary-counter forest of perfect reduction trees — so admitting an asset is **path copying, not mutation**: $O(\log n)$ new merge nodes (amortized $O(1)$) over live mods, no existing node re-registered or re-executed, sibling subtrees physically reused (trace, provenance and all), and consumers hold one stable total mod throughout. Removal stays non-structural: trade the position to qty 0.

(psac:run-dynamic-book-demo) on the same five-asset book:

```

firm P&L over 5 assets: 1215000
tick AAPL -> 1265000: 4 nodes re-ran (leaf-to-root path + top): pnl-node[AAPL], firm-pnl-merge, firm-pnl-merge, firm-pnl-top
add TSLA -> 1275000: 0 nodes re-ran (insertion is additive: new nodes only)

```

A tick re-runs only the leaf-to-root path — $O(\log n)$, where the flat universe’s aggregates each re-read all n positions. Adding TSLA (qty 10, basis 24000, price 25000: P&L +10000) re-runs *zero* existing nodes: the new pnl node and merge nodes compute once at registration. This is reuse-by-structure — the part of SAC trace memoization this workload needs, obtained by shaping the graph like a persistent data structure instead of timestamping the trace.

11. Parallel execution: the same graph, more workers

Everything above runs unchanged in parallel; results and bills are identical by construction, because the scheduler only reorders *independent* work.

Across nodes — propagate-parallel! drains dirty nodes in (stratum, height) waves on an lparallel kernel (which does work stealing internally). By the height invariant, same-level nodes never read each other’s outputs, so each wave runs concurrently with a barrier between levels. A tick storm across the whole book — every pnl-node[T] dirty at once — is exactly this shape:

```

(dolist (tick storm) (tick! u (car tick) (cdr tick) :propagate nil))
(propagate-parallel! :workers 8) ; same values, same bill as propagate!

```

Inside a node — par and par-map fork-join within a single thunk, for the case level parallelism cannot touch: one dirty node with a heavy body (think a Monte-Carlo VaR node mapping over paths):

```

(register-read (list params)
  (lambda (p)
    (write! var (percentile 99 (par-map (lambda (path) (revalue p path))
                                         paths))))
  :name "mc-var-node" :cost 50)

```

Dynamic state (bill, blame, labels) is conveyed to workers explicitly; spawning stops beyond $\text{ceil}(\log_2 \text{workers}) + 2$ nested pars (tunable via *par-max-depth*), falling back to sequential in the caller. Nested adaptive-reads created in branches are tagged :par in the trace (RSP-lite).

Measured on 8 workers (bench-parallel / bench-par-within, 64 heavy nodes / items): **~6.3x** both across nodes and within one node. Policy stratification, billing determinism, and provenance all survive parallel execution — the wave order only permutes log entries, never values or bills.

12. Formal guarantees (Lean 4, no sorries)

Theorem	File	What it guarantees for the demo
propagate_correct	Basic.lean	incremental propagation $\equiv$ from-scratch: every number after a tick equals a full re-price
support_sound	Support.lean	full (non-selective) supports are honest: inputs outside the support cannot change a view (data-dependence slice; control

Theorem	File	What it guarantees for the demo
		dependence is covered by tests)
charge_conserve, bill_conserve, blame_head_lowest	Cost.lean	bills are exactly conserved — split work sums back to total work, remainder to the lowest principal
step_comm, eval_perm, eval_blocks_comm, waves_WF, propagate_waves_correct	ParLevel.lean	parallel propagation and fork-join may reorder independent work without changing any result (well-formedness needed only of the canonical order)
scenario_observe	Scenario.lean	a what-if reads exactly the from-scratch value of the hypothetical world
scenario_roundtrip	Scenario.lean	scenario rollback restores the base world exactly
scenario_private	Scenario.lean	a scenario writing outside an observer's support is invisible to that observer

The Lean model covers the sequential trace semantics and the order-irrelevance arguments; that the runtime establishes their premises (heights, single-writer discipline) is covered by the test suite and a CL-vs-Lean differential harness.

13. Run it yourself

```
devcontainer up --workspace-folder .
devcontainer exec --workspace-folder . bash scripts/test.sh           # rove suite
devcontainer exec --workspace-folder . bash scripts/build-model.sh    # Lean proofs + oracle
devcontainer exec --workspace-folder . bash scripts/diff-test.sh      # CL vs Lean differential
```

then, in a REPL with the psac system loaded: (psac:run-portfolio-demo) and (psac:run-dynamic-book-demo).